Promise Pals: demo video script

Length: about 3 minutes 15 seconds (Devpost allows 2 to 5). Read calmly. **Bold** = what you say. 🎬 = what's on screen.

Before you record: two logged-in phones (sign up the first phone LAST, so it doesn't wait 30 seconds and get a practice partner), Fitbit synced, Do Not Disturb on. No second phone? Use **See a demo**: same flow with labelled sample people.

---

1. The problem (0:00 to 0:35)

🎬 Landing page: the two characters walking in, "Team up!"

"Hi, we're [team name], and this is Promise Pals.

Lots of people want a partner to keep them on track with a health goal, like their physio exercises or a daily walk. But there aren't many ways to find one. Your friends have different goals, and most apps just give you a step counter and leave you on your own.

And it matters. Up to half of people don't keep up their home exercises. Research across 148 studies shows strong social ties are linked to 50% better survival odds, and across 138 studies, tracking progress works better when you report it to someone else. People do better with a partner. They just can't find one."

2. Finding a partner (0:35 to 1:15)

🎬 Sign up, then the profile: first name, community list, **How do you move?**

"Promise Pals fixes that. You sign up, pick your Calgary community from the City of Calgary's open data, and, if you want, tell us how you move: walking, a wheelchair, a walker, or chronic pain."

🎬 Make a promise, then the **Nearby board**.

"Then you see real people near you who are looking right now: their first name, their community, how far away they are, and their goal. People who move like you show up first."

🎬 Tap **Ask … to team up**. The second phone shows the request. Tap **Team up**. Both phones say **"You're a team!"**

"I ask someone to team up, their phone gets it within seconds, they say yes, and we're partners. If nobody's around, you can invite a friend with a link, and a labelled practice partner steps in so you're never stuck waiting."

3. Real proof (1:15 to 1:55)

🎬 Today screen, then **Check in with a photo**, then the partner confirming on the second phone.

"Every day you check in, with real proof. A photo of what you did, checked by AI before your partner sees it, and your partner confirms it."

🎬 **Check in with Fitbit**: steps, workout and active minutes appear.

"Or your Fitbit. We read your real data through Google's Health API: your steps, workouts your Fitbit spots on its own, and active minutes from your heart rate, so a wheelchair workout counts as much as a walk. If your Fitbit shows nothing, the check-in doesn't count."

4. Accountability (1:55 to 2:15)

🎬 Demo mood buttons: Annoyed, Mad, Furious, then Sad. Then Progress: badges.

"Let your partner down, and your character notices: annoyed, then mad, then furious. But rough days are allowed and never break your streak. Keep going and you earn badges and unlock new looks."

5. Accessibility (2:15 to 2:45)

🎬 Landing page, accessibility section: flip **Bigger text**, **Stronger contrast**, **Less motion** live.

"We built this for the people most apps leave out. Find someone who moves like you. Proof from heart rate, not just steps. These switches change the whole app instantly: bigger text, stronger contrast, less motion. It works with screen readers and a keyboard, every character's mood is described in words, text is at least 18 pixels, colours meet the WCAG AA standard, and every screen passes automated accessibility checks with zero issues."

6. How it's built (2:45 to 3:10)

🎬 The GitHub repo or the code.

"It's a Next.js web app on Vercel, so it works on any phone with nothing to download. Auth0 handles logins, so we never store a password. Our data lives in Postgres on Tiger Data. Fitbit data comes from Google's Health API, communities from the City of Calgary's open data, and Google Gemini checks every goal and photo, with a 'Why?' panel showing how it decided. We wrote the code with Claude Code, and our team designed it, made every decision, and tested it."

7. Close (3:10 to 3:15)

🎬 Both phones on Today.

"Promise Pals: find a health partner near you, and keep each other going. Thank you."

Cheat sheet: how the code works

Use this for Q&A, or to explain the code yourself. Each answer follows the pattern "we used X to do Y". File names are in `code style`, so you can open them and point.

The big picture

- **"What is it built with?"**
  - **Next.js 16** with **React 19** and **TypeScript**, a framework for making websites that act like apps. One project holds both the screens and the server.
  - **Tailwind CSS** for styling.
  - **Vercel** hosts it. We deploy with one command: `npx vercel deploy --prod`.
- **"Why a website and not an app?"** Anyone can open it on any phone in seconds, and it can be added to the home screen like an app. A native app comes next, mainly to read Apple Health.
- **How the screens stay live:** every phone asks the server "what's new?" every 3 seconds (`REFRESH_EVERY_MS = 3000` in `components/App.tsx`). That's how a team-up request or a check-in shows up on your partner's phone almost instantly.

Where the code lives

- `app/` holds the pages and the server routes. Every server action is a file like `app/api/checkin/route.ts`, so keys and secrets stay on the server and never reach the phone.
- `components/` holds the screens: the landing page, Today, the Nearby board, Progress and Me.
- `lib/` holds the logic:
  - `lib/rules.ts`: every rule in one place (streaks, the mascot's moods, points, badges).
  - `lib/nearby.ts`: the Nearby board.
  - `lib/fitbit.ts`: reading Fitbit data.
  - `lib/moderation.ts` and `lib/gemini.ts`: the AI checks.
- `db/schema.sql` lists every database table.

Accounts and logins

- **"How do logins work?"** We used **Auth0**. When you sign up, our server passes your email and password straight to Auth0, and Auth0 checks them. **We never store your password, and we don't keep your email.**
- After you log in, the phone gets a random token. We store only a scrambled version of it, a **SHA-256 hash**, so even a stolen database copy can't be used to log in.

The Nearby board (`lib/nearby.ts`)

- **"How do you find people near me?"** One database query finds everyone who is:
  - looking right now
  - seen in the last 3 days
  - not your old partner
  - not someone who already said "not now" to you
- **Distance:** we take the centre point of each Calgary community from the City's open data and measure between them. **We never know anyone's real location**, only their community.
- **Order:** people who move like you come first, then the closest, then the same kind of goal.
- **Asking and accepting:** "Ask to team up" saves a row in a `team_requests` table. Your partner's phone sees it on the next 3-second refresh. When they tap yes, we use a **database transaction that locks both people** (`FOR UPDATE`), so even if two people say yes at the same moment, nobody ends up with two partners.
- **Invite links:** each person gets a short random code. Whoever makes a promise through that link is paired with you straight away (`joinByInvite`).

Check-ins and proof

- **Photos:** the phone shrinks the photo first (`shrinkPhoto` in `components/Photos.tsx` makes it a small JPEG), so it uploads fast on bad Wi-Fi. The server sends it to **Gemini**, which says whether it's safe and describes it in a few words. Only then can your partner see it. Photos are only ever shown to you and your partner.
- **Fitbit (`lib/fitbit.ts`):**
  - You connect once with Google's sign-in (**OAuth**). We keep a long-lived key, never your Google password.
  - When you check in with Fitbit, we ask **Google's Health API** for today's **steps**, **workouts** (the `exercise` data type) and **active zone minutes** (from heart rate). We add them up and save them as proof.
  - "Check in with Fitbit" is refused unless Fitbit shows real activity today, so it can't be faked.
- **Points (`checkinPoints` in `lib/rules.ts`):** check in +10, partner confirms +5, Fitbit proof +20, an honest rough day +5. Points unlock character extras, and the server checks you've earned them, so nobody can cheat by editing the app.

The mascot (`mascotFor` in `lib/rules.ts`)

- **"How does it decide its mood?"** It's plain rules, no AI. It looks at:
  - whether you checked in today
  - how late you are past your check-in time

    - how many days you've gone quiet
- One quiet day makes it mad, two or more makes it furious. A rough day you told your partner about makes it sad, never angry. Gentle mode never gets angry at all.
- After **2 silent days** (`SILENT_DAYS_BEFORE_NEW_MATCH = 2`), the waiting partner can get someone new.
- The characters are drawn pixel by pixel in code (`lib/avatar.ts`), ported from our teammate's character design, so every outfit and mood is generated live, not saved as images.

AI (Gemini): what it does and doesn't do

- **Models:** `gemini-3.5-flash-lite`, with `gemini-3.1-flash-lite` as a backup if the first is busy.
- **What it does:**
    - checks a **goal** (safe, not a weight-loss goal, and rewrites it in plain words for your partner, without personal details)
    - checks your **"why it matters"** line
    - checks **photos**
- **What it never does:** chat, give health advice, choose your partner, or decide your streak. Those are all plain rules we wrote.
- **Transparency:** every AI decision has a **"Why?" panel** showing what it checked and what it decided.
- **If Gemini is down:** goal checks still let you continue, and photos ask you to try again or use Fitbit. The app never breaks.

Accessibility (how each one is built)

- **Real buttons and labels:** every tap target is a real `<button>` or link with a label. Icon buttons have `aria-label`s. Each character has a text description of its mood (`screenReaderText`).
- **Never colour alone:** the week grid uses ✓ ~ × symbols and hidden words as well as colours.
- **Big text and contrast:** body text is 18 pixels, and colours were checked against WCAG AA.
- **Easy reading switches (`lib/reading.ts`):** they set a tag on the page, like `data-text="large"`, and our CSS reacts to it. The choice is saved on the phone and applied **before the page draws**, so there's no flash of small text.
- **Less motion:** we respect the phone's own "reduce motion" setting, plus our own switch. Every animation checks `wantsLessMotion()`.
- **How we test it:** we run **axe-core**, the industry-standard accessibility checker, over every screen in an automated browser. It reports **0 issues**.

Privacy and safety

- **Delete my account and data** (`app/api/account/delete/route.ts`) removes your profile, photos, goals, Fitbit link and login from our database. Your partner keeps empty check-in marks so their streak still adds up, with nothing about you in them.
- No meet-ups through the app, no free chat, 18+ only, and report or leave in one tap.

Testing

- **120 server checks** (`scripts/smoke-test.mjs`): they sign up real test accounts, pair them through the board, check in, confirm, delete, and more, against a throwaway test database.
- **38 rule tests** (`lib/rules.test.ts`, run with `npm test`): streaks, moods, points, Fitbit parsing.
- **Browser tests with accessibility checks** on every screen.

Likely judge questions

- **"Who pays?"** Calgary physio clinics and community programs. Their patients don't do their home exercises, and Promise Pals makes them more likely to. It costs cents per user to run.
- **"What if nobody's on the board?"** We launch through clinics and community centres so the board is full from day one, and invite links fill the gaps.
- **"What about iPhone users without a Fitbit?"** A photo works on any phone. Apple only lets App Store apps read Apple Health, so a native app is our next step.
- **"Is this real data?"** Yes: real Fitbit data, real City of Calgary communities, real accounts. The **See a demo** mode uses sample people and says so on screen.
- **"How did you use AI?"** Claude Code wrote the code, with our team designing, deciding and testing every step. Gemini only checks goals and photos inside the app, and every decision has a Why panel.
- **"How is it safe with strangers?"** First names only, no chat, no meet-ups, AI checks before anyone sees anything, 18+, report and leave in one tap.

Fill in before recording:

- [ ] Your team name (Part 1)
- [ ] Which tools your teammates used for the characters and mockups. Add one line to Part 7, for example: "Our teammate designed the characters in [tool]."